Design Flaws in Modern Wi-Fi Security

Analysis of three Wi-Fi attacks: KRACK, Fragment/Forge and AirSnitch

- Candidate: Cioni Jacopo (MAT. 581651)
- Course: ICT Risk Assessment 2025/2026
- Supervised by: Prof. Deri, Prof. Baiardi and Prof. Sammartino

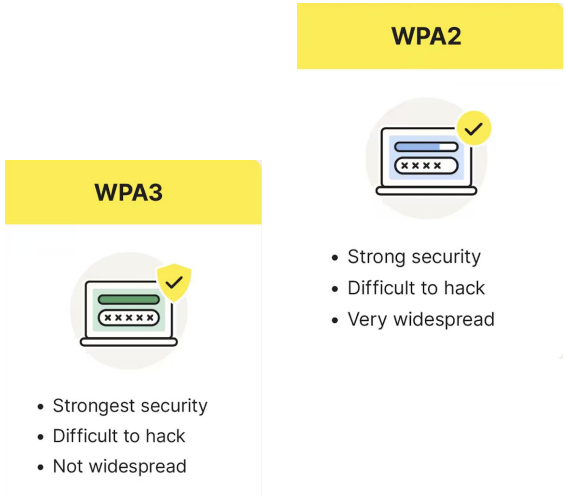
Is Wi-Fi secure today ?

Security failures are often caused by protocol logic, not by weak cryptography.

Key points in the security of wifi networks:

1. strong cryptography provided by WPA2 & WPA3 protocols
2. formally verified handshakes
3. client isolation techniques

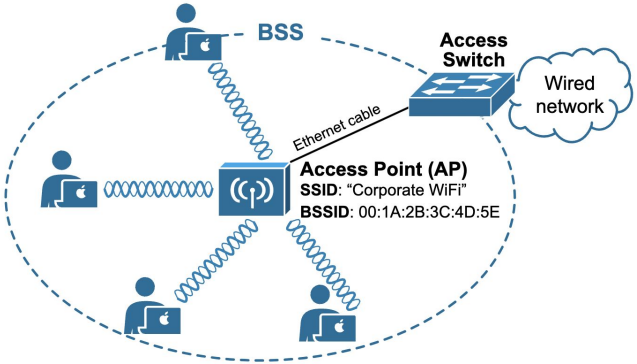
but vulnerabilities continue to emerge…

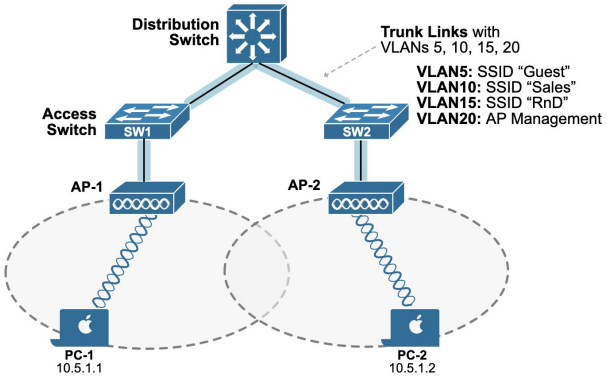


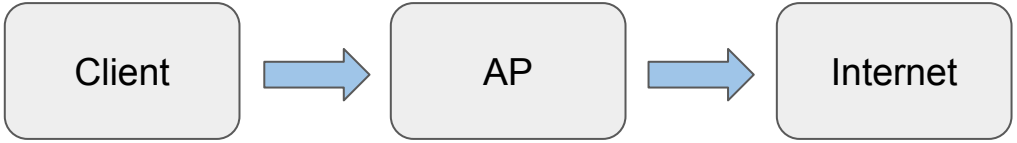
Wi-Fi Architecture

At the core is a router or gateway. It connects physical Access Points, which support Wi-Fi communication, to upstream networks (the Internet), and to services like DNS and DHCP servers.

A Wi-Fi network is identified by a user-firendly string called the Service Set Identifier (SSID), which allows clients to discover the network. A BSS consists consists of a central AP along with connected clients that all operate on the same frequency band. A BSS is identified by a unique BSS identifier (BSSID), which is typically the MAC address of the AP.







UNIVERSITÀ DI PISA

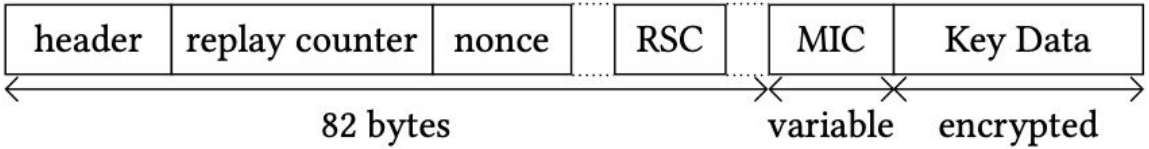
Layers of Security

Different layers of Wi-Fi design can introduce vulnerabilities.

LAYER	ATTACK
Handshake: responsible for authenticating the client and AP and establishing session keys through protocols (e.g. WPA2/WPA3 4-way handshake).	**KRACK:** exploits the retransmission of handshake messages to force key reinstallation, causing packet manipulation.
Frame Processing: handles the processing of frames, including aggregation and fragmentation.	**Fragment & Forge:** inject packets or combine fragments from different frames, enabling data injection.
Network Isolation: manage isolation mechanisms between clients connected to the same AP.	**AirSnitch:** client isolation i bypassed by manipulating frame forwarding behaviour, enabling attacks between clients.

KRACK Attack (1)

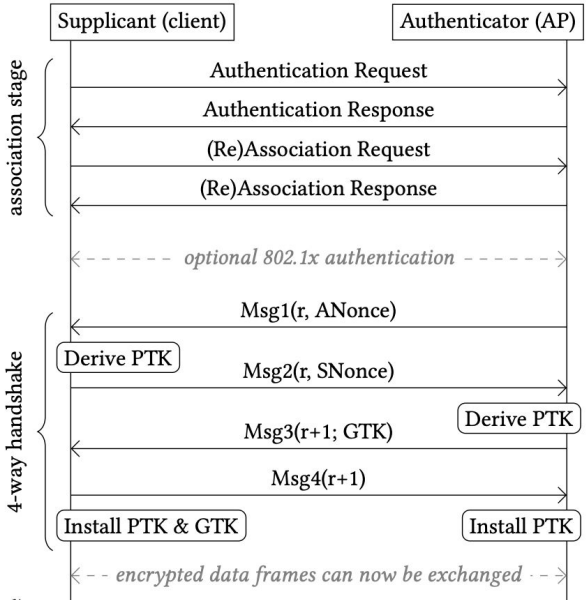
The 4-way-handshake allows you to authenticate the client and the AP and derive both the PTK and GTK session keys. The GTK is shared by all clients on the network with the same BSSID and is used for broadcast communications.



EAPOL frames carry the key exchange messages of the WPA2/WPA3 4-way-handshake. Most important fields:

- **replay counter:** incremental number used to avoid replay attacks. +1 everytime AP sends a message
- **nonce:** random number used to generate the PTK key
- **MIC:** cryptographic signature that verifies the integrity of the message
- **key data:** contains distributed keys (e.g. GTK)

KRACK Attack (2)



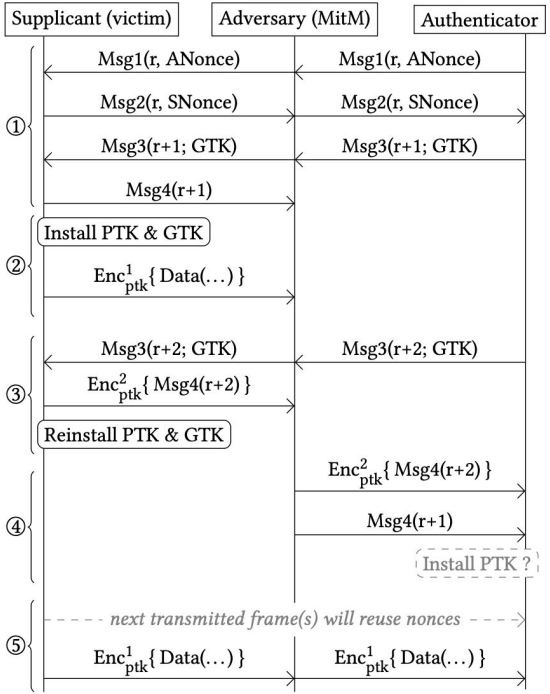
Message exchange:

1. **Msg1:** the AP sens ANonce, a random value used to derive the session key
2. **Msg2:** the client derive the PTK and generates SNonce, sends it to the AP. Both devices can now derive the PTK.
3. **Msg3:** the AP confirms the key negotiation and sends the GTK.
4. **Msg4:** the client acknowledges the installation of the keys.

The vulnerability occurs because retransmitted Msg3 can cause the client to reinstall the same key, resetting the nonce and breaking the security of the encryption.

KRACK exploits the retransmission of EAPOL Msg3, forcing the client to reinstall the same encryption key and reuse the nonce!!

KRACK Attack (3): Impact of Nonce Reuse



The attacker positions themselves as a Man-in-the-Middle (MitM) between the client and the AP. By blocking or delaying handshake messages, the attacker forces AP to retransmit EAPOL Msg3.

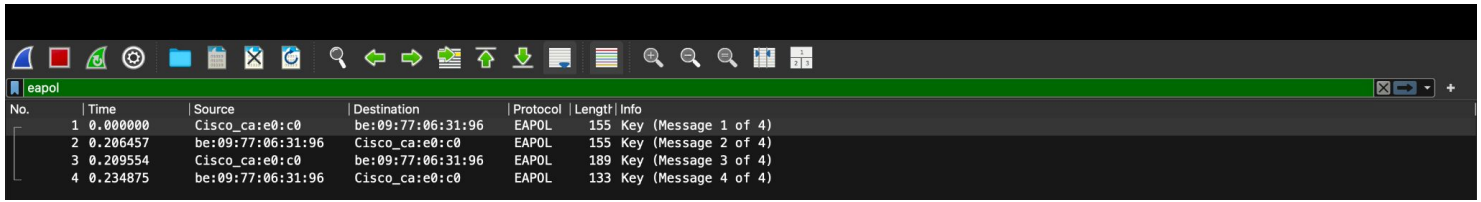
When the client receives the retransmitted Msg3, it reinstalls the same encryption key. This causes:

- nonce reuse
- the keystream becomes predictable
- encrypted packets can be decrypted or manipulated

Possible attacks: packet replay, packet decryption, packet injection.

The attacker does not break the encryption directly. Instead, they manipulate the handshake to force key reinstallation !!

KRACK Attack (4): Inspection (Msg1)



The 4-way-handshake

```
Key Descriptor Type: EAPOL RSN Key (2)
[Message number: 1]
Key Information: 0x008a
    .... .... .... .010 = Key Descriptor Version: AES Cipher, HMAC-SHA1 MIC (2)
    .... .... .... 1... = Key Type: Pairwise Key
    .... .... ..00 .... = Key Index: 0
    .... .... .0.. .... = Install: Not set
    .... .... 1... .... = Key ACK: Set
    .... ...0 .... .... = Key MIC: Not set
    .... ..0. .... .... = Secure: Not set
    .... .0.. .... .... = Error: Not set
    .... 0... .... .... = Request: Not set
    ...0 .... .... .... = Encrypted Key Data: Not set
    ..0. .... .... .... = SMK Message: Not set
Key Length: 16
Replay Counter: 2
WPA Key Nonce: 7e9e4a5a45118061996946a2cde36060dbd7cbab3c655e3c0354be29d5a87521
Key IV: 00000000000000000000000000000000
WPA Key RSC: 0000000000000000
WPA Key ID: 0000000000000000
WPA Key MIC: 00000000000000000000000000000000
WPA Key Data Length: 22
WPA Key Data: dd14000fac0477e76a18bc306a97ecb82fcc82f050ad
```

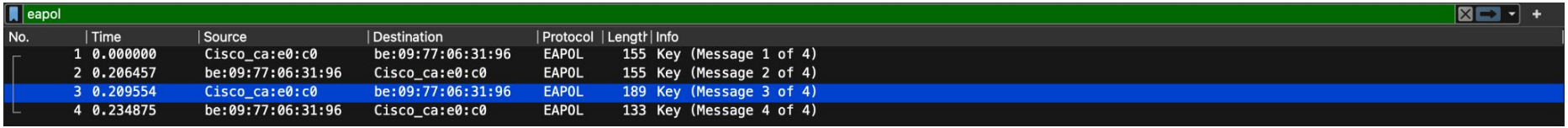
If “set” is active, this flag means “install the derived key” (not possible on Msg1).

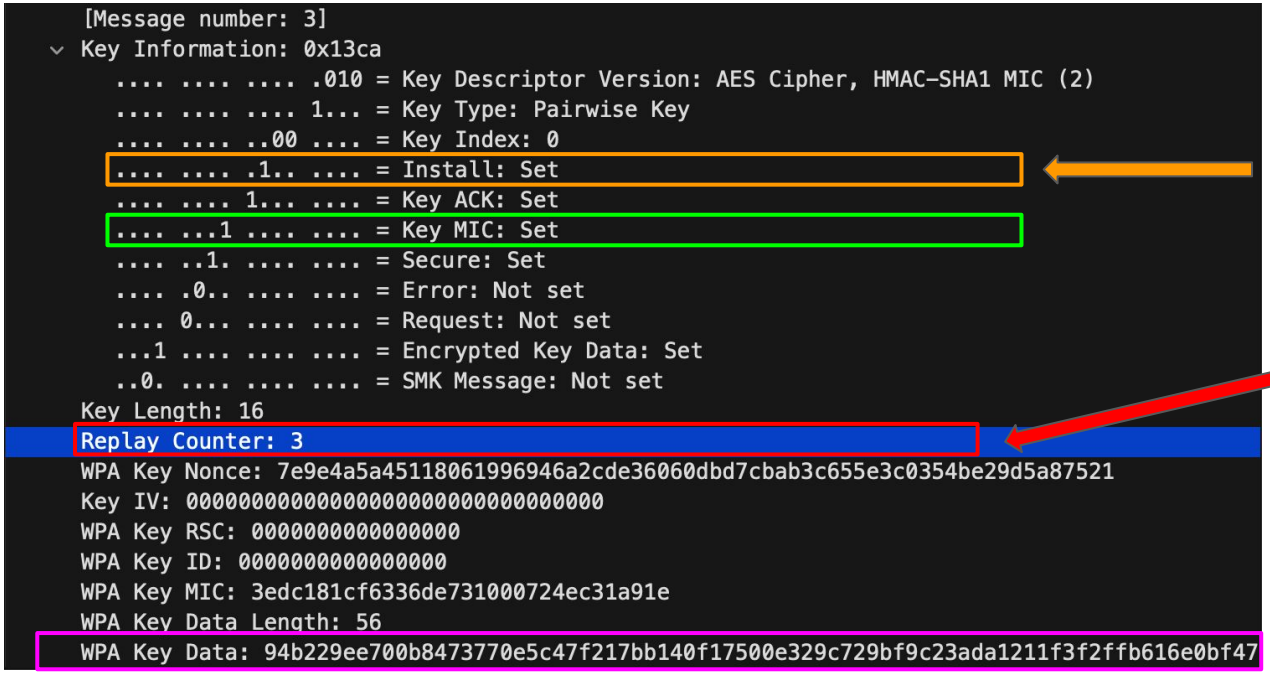
Replay counter set to 2. It will be +1 on the next Msg from the AP.

This is the ANonce generated by the AP. It is used to derive the PTK.

UNIVERSITÀ DI PISA

KRACK Attack (4): Inspection (Msg3)





Now it is active. Client can install PTK key.

+1

GTK here

KRACK Attack (4): Scapy

```
frame = (
    RadioTap()
    / Dot11(
        addr1="ff:ff:ff:ff:ff:ff",   # destination
        addr2="00:11:22:33:44:55",   # source
        addr3="00:11:22:33:44:55"    # BSSID
    )
    / LLC()
    / SNAP()
    / IP(dst="192.168.1.1")
    / TCP(dport=80)
    / Raw(load=b"DemoPayload")
)

frame.show()
```

Using Scapy, I can build a Wi-Fi frame programmatically.

An attacker can modify frame fields such as addresses or payload destinations.

```
frame.addr1 = "aa:bb:cc:dd:ee:ff"
frame[IP].dst = "10.0.0.5"
frame.show()
```

Once built, the frame becomes a real byte sequence:

```
[5]: raw_bytes = bytes(frame)
     print(raw_bytes[:40])
     print("Frame length:", len(raw_bytes))

     b'\x00\x00\x08\x00\x00\x00\x00\x00\x08\x00\x00\x00\xaa\xbb\xcc\xdd\xee\xff\x00\x11"3DU\x00\x11"3DU\x00\x00\xaa\xaa\x03\x00\x00\x00\x08\x00'
     Frame length: 91
```

KRACK Attack (5): conclusions & countermeasures

KRACK does not break WPA itself, but exploits weakness in its implementation of the 4-way-handshake.

The KRACK attack was fixed by updating client implementations so that retransmitted handshake messages do not reinstall the encryption key. Additionally, using HTTPS mitigates most practical attacks, because the attacker cannot decrypt application-level traffic.

Fragment & Forge Attack (1)

FragAttacks are a set of vulnerabilities discovered in 2021 that exploit weakness in how Wi-Fi devices handle fragmented frames and certain frame aggregation mechanisms. This attack affects multiple Wi-Fi standards (WEP, WPA2, WPA3).

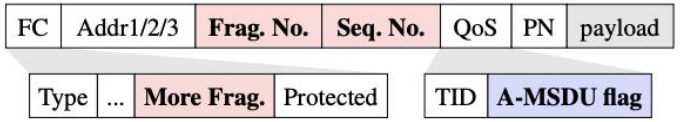
Packet fragmentation is used to improve Wi-Fi performance. This allows you to divide a large frame into multiple fragments.

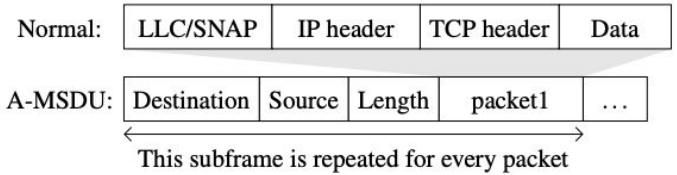
Where is the problem ?

The receiver does not check that all fragments use the same key and therefore the attacker can mix fragments from different frames by constructing valid fake packets !!

Fragment & Forge Attack (2): frame fields

- **Fragment Number**: identifies fragment position
- **Sequence Number**: groups fragments of the same packet
- **More Fragments flag**: indicates if more fragments follow
- **Protected flag**: indicates encryption
- **A-MSDU flag**: indicates aggregated frames

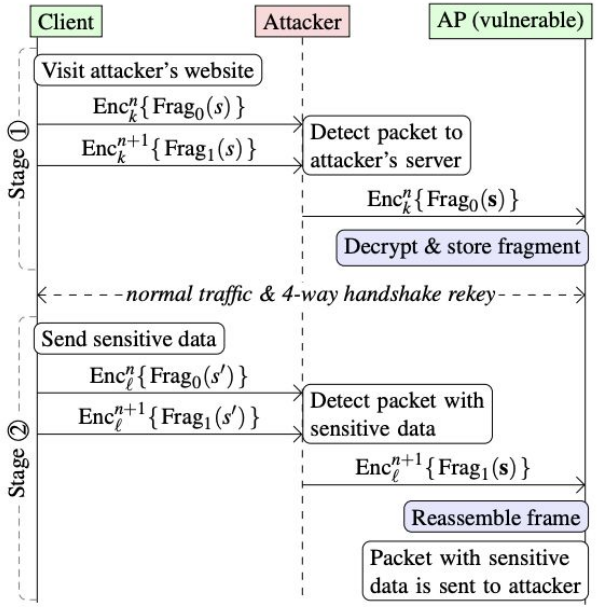




The sequence number groups fragments, while the fragment number defines their order. If these fields are not properly validated, an attacker can inject malicious fragments !!

In A-MSDU aggregation, multiple packets are combined into a single Wi-Fi frame. Each subframe contains its own destination and payload.

Fragment & Forge Attack (3): attack flow



Stage 1 - preparation: in the first stage, the attacker prepares the attack by injecting or capturing a fragment that will be later reused.

1. The client visits a website controlled by the attacker.
2. The client sends fragmented traffic.
3. The attacker intercepts a fragment.
4. The vulnerable AP accepts the fragment and stores it.

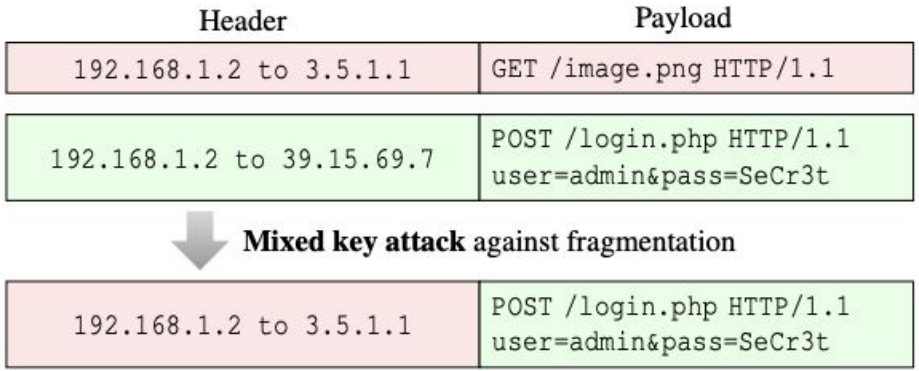
Stage 2 - exploitation: in the second stage, the attacker combines fragments from different packets, causing the AP to reconstruct a packet containing sensitive data.

1. The client sends sensitive data (login).
2. The attacker intercepts another fragment.
3. The AP combines different fragments.
4. The resulting packet is sent to the attacker.

Fragment & Forge Attack (3): attack flow (2)

The attacker mixes fragments from two different packets: one controls the destination, the other contains sensitive data.

Fragments are reassembled based on sequence number and fragment number, but they are not always validated.



Red ones → to the attacker
Green one → contains sensitive data

The vulnerability arises because fragments are reassembled without verifying that they belong to the same original packet !!

Fragment & Forge Attack (4): demo

```python
from scapy.all import *

# Fragment 1
frag1 = RadioTap()/Dot11(
    type=2, subtype=0,
    addr1="ff:ff:ff:ff:ff:ff",
    addr2="00:11:22:33:44:55",
    addr3="00:11:22:33:44:55",
    SC=16  # same sequence
)/LLC()/SNAP()/IP(dst="192.168.1.1")/Raw(load=b"FRAG_1")

# Fragment 2
frag2 = RadioTap()/Dot11(
    type=2, subtype=0,
    addr1="ff:ff:ff:ff:ff:ff",
    addr2="00:11:22:33:44:55",
    addr3="00:11:22:33:44:55",
    SC=16  # same sequence
)/LLC()/SNAP()/IP(dst="39.15.69.7")/Raw(load=b"FRAG_2_SENSITIVE")

# Salva in file
wrpcap("frag_demo.pcap", [frag1, frag2])
```

header wifi 802.11

Here we intentionally create two frames with the same sequence and fragment numbers. This simulates a scenario where a vulnerable device might incorrectly associate them.

These 3 addresses help identify source, destination and the network context (receiver, transmitter, BSSID)

Sequence control contains both the sequence number and the fragment number. They have the same value so they appear logically related.

Fragment & Forge Attack (5): inspection

Then inspect the generated PCAP in wireshark to verify the relevant fields and compare the frames.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.10.3	192.168.1.1	IPv4	66	IPv6 hop-by-hop options [Malformed Packet]
2	0.000782	172.20.10.3	39.15.69.7	IPv4	76	IPv6 hop-by-hop options [Malformed Packet]

Different destinations !

```
> Frame 1: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits)
> Radiotap Header v0, Length 8
  802.11 radio information
v IEEE 802.11 Data, Flags: ........
    Type/Subtype: Data (0x0020)
  > Frame Control Field: 0x0800
    .000 0000 0000 0000 = Duration: 0 microseconds
  > Receiver address: Broadcast (ff:ff:ff:ff:ff:ff)
  > Transmitter address: CIMSYS_33:44:55 (00:11:22:33:44:55)
  > Destination address: Broadcast (ff:ff:ff:ff:ff:ff)
  > Source address: CIMSYS_33:44:55 (00:11:22:33:44:55)
  > BSS Id: CIMSYS_33:44:55 (00:11:22:33:44:55)
    .... .... .... 0000 = Fragment number: 0
    0000 0000 0001 .... = Sequence number: 1
```

```
> Frame 2: Packet, 76 bytes on wire (608 bits), 76 bytes captured (608 bits)
> Radiotap Header v0, Length 8
  802.11 radio information
v IEEE 802.11 Data, Flags: ........
    Type/Subtype: Data (0x0020)
  > Frame Control Field: 0x0800
    .000 0000 0000 0000 = Duration: 0 microseconds
  > Receiver address: Broadcast (ff:ff:ff:ff:ff:ff)
  > Transmitter address: CIMSYS_33:44:55 (00:11:22:33:44:55)
  > Destination address: Broadcast (ff:ff:ff:ff:ff:ff)
  > Source address: CIMSYS_33:44:55 (00:11:22:33:44:55)
  > BSS Id: CIMSYS_33:44:55 (00:11:22:33:44:55)
    .... .... .... 0000 = Fragment number: 0
    0000 0000 0001 .... = Sequence number: 1
```

Both frames expose the same fragment number and the same sequence number. Vulnerable implementation could treat them as related.

Fragment & Forge (6): conclusions & countermeasures

This attack exploit the fact that some devices may reassemble fragments without properly validating that they belong to the same original packet.

In this demonstration we can see the protocol level inconsistency that such an attack relies on.

The main defence is to **enforce strict validation of fragments**: devices must ensure that all fragments belong to the same packet and context. Using end-to-end encryption helps mitigate the impact of these attacks.

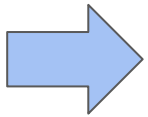
FragAttacks are not caused by broken encryption, but by design and implementation flaws in how devices process Wi-Fi frames. Bot KRACK and FragAttacks show that security does not depend only on encryption but also on correct protocol implementation !!

AirSnitch Attack (1)

AirSnitch (2026) is a covert channel attack that enables communication between isolated Wi-Fi clients. Some details:

- does not break encryption (WPA2/WPA3)
- uses Wi-Fi frames behavior as a signaling channel
- allows indirect communication through the AP

In many networks, client isolation is used to prevent devices from communicating with each other. Even if direct communication is blocked, clients can still exchange information through observable frame patterns.



AirSnitch exploits the fact that some low-level Wi-Fi behaviours are still observable by other clients.

Even without direct communication, a hidden channel is created !!

AirSnitch Attack (2): attacker assumptions & goals

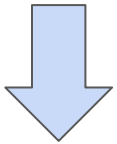
We consider an *in-network* attacker who has access to the Wi-Fi infrastructure by connecting to an open SSID or by using its own credentials to connect to an encrypted SSID. The attacker is also capable of simultaneously transmitting and receiving Wi-Fi signals to and from both the victim and their associated AP.

The attacker aims to bypass client isolation. It seeks to intercept a victim user's traffic and inject traffic towards others users and/or AP.

THE IDEA: bypass client isolation exploiting the shared GTK.

AirSnitch Attack (3): abusing GTK

The GTK key is shared by all clients on the network with the same BSSID and is used for broadcast/multicast communications. If the attacker obtains the GTK key (just associate with the BSSID) he can construct a group frame encrypted with GTK but containing unicast payload.



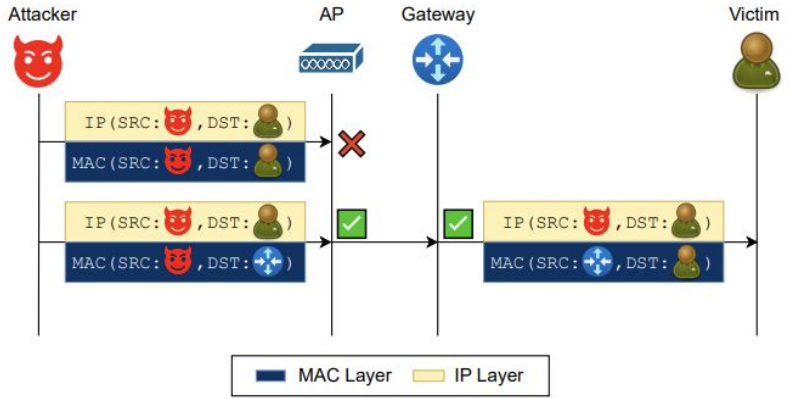
The victim accepts that frame because it is valid at the Wi-Fi level. The AP cannot block it because the frame does not go through direct forwarding, but is delivered “over the air” as group traffic.

AirSnitch Attack (4): gateway bouncing

This diagram shows why client isolation can fail:

- In the first case, direct communication between two clients is blocked. This traffic would normally use PTK and is correctly filtered.

- In the second case, the attacker sends a frame (using GTK) that is not treated as a direct client-client traffic. The AP forwards it because the MAC destination appears valid. Since all clients know the GTK, they can decrypt these frames.



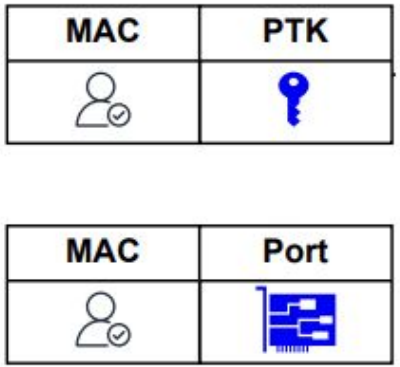
Many router treat client-gateway traffic as legitimate and do not distinguish that the real end is another client !!

AirSnitch Attack (5): port stealing

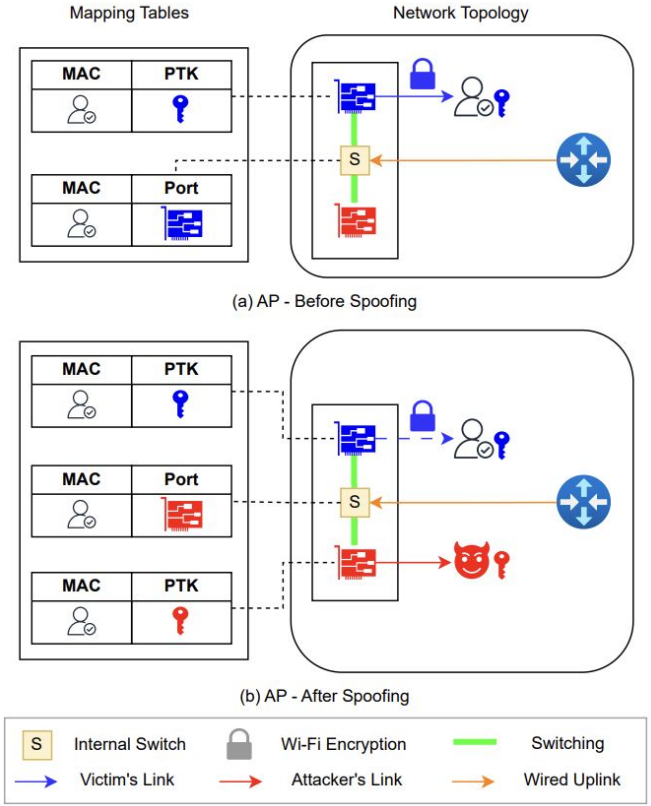
Port stealing means tricking the AP into associating the victim's MAC address with the attacker's virtual port. The AP behaves, from a logical point of view, like a switch and keeps a table that associates:

- MAC address with the logical port
- MAC address with the PTK

If the attacker manages to make the AP believe that the victim's MAC is on another port, then the traffic destined for the victim is forwarded to the attacker.



AirSnitch Attack (5): port stealing (2)



- **AP - Before Spoofing (Normal situation)** The victim is associated with his virtual door and the internal table of the AP is correct. The traffic from the gateway is successfully forwarded to the victim and the encryption uses the victim's key. Everything is consistent.
- **AP - After Spoofing** The attacker authenticates on a different port and spoofs the victim's MAC address. The AP updates its internal mapping and associates the victim's MAC with the attacker's port and key. As a result, packets destined for the victim are forwarded to the attacker and encrypted with the attacker's PTK.

AirSnitch Attack (6): scenarios

AirSnitch enables several practical attack scenarios:

1. The attacker can intercept traffic that should only be visible to another client. Using port stealing, traffic intended for a victim can be redirected to the attacker.
2. The attacker can inject packets into the network. By crafting frames encrypted with the GTK, the attacker can send data that is accepted by other clients.
3. The attacker can position themselves as man-in-the-middle. By combining redirection and injection, the attacker can both receive and modify traffic.

Additionally, AirSnitch allows covert communications between isolated clients and this can be used to exchange information without being detected.

AirSnitch enables both passive and active capabilities !!

AirSnitch Attack (7): demo

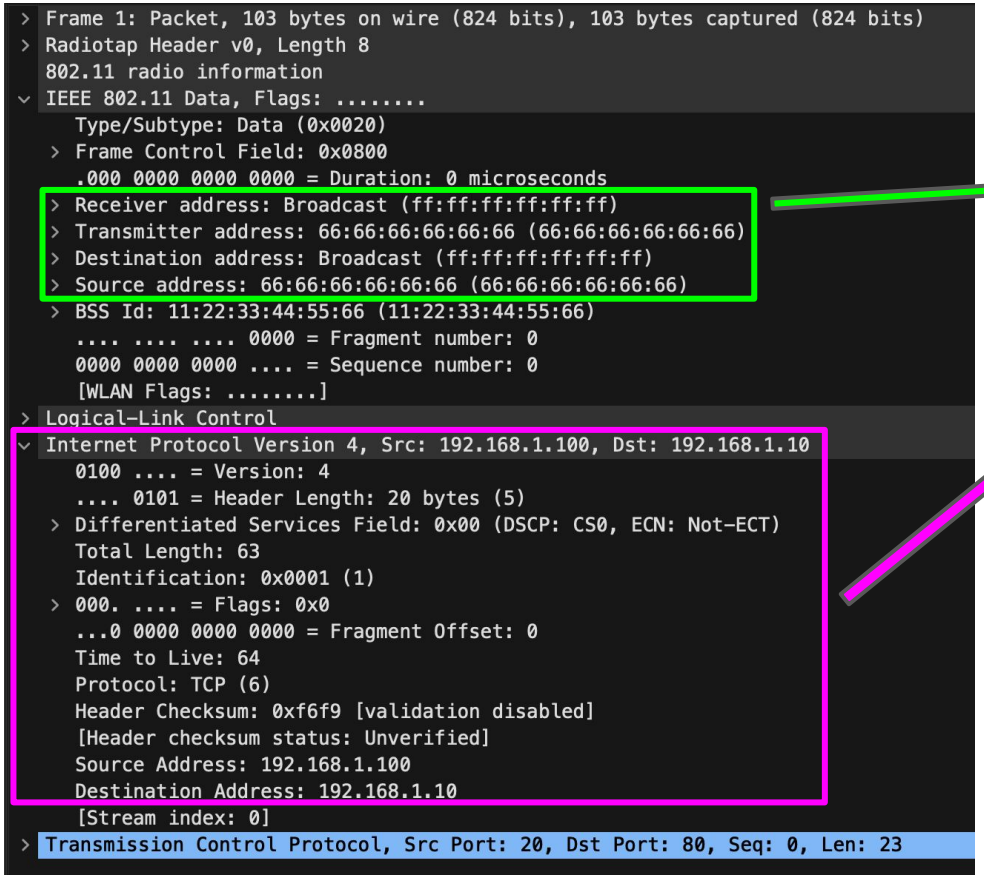
```python
from scapy.all import *

frame = RadioTap()/Dot11(
    type=2, subtype=0,
    addr1="ff:ff:ff:ff:ff:ff",   # broadcast → GTK
    addr2="66:66:66:66:66:66",   # attacker
    addr3="11:22:33:44:55:66"    # AP
)/LLC()/SNAP()/IP(
    src="192.168.1.100",
    dst="192.168.1.10"           # vittima
)/TCP()/Raw(load=b"FAKE_DATA_FROM_ATTACKER")

wrpcap("airsnitch_demo.pcap", [frame])
```

The frame is broadcast at the MAC layer, so it is accepted and decrypted by all clients using the shared GTK. However, the IP layer targets a specific victim and this allows the attacker to inject traffic without direct communication.

AirSnitch Attack (8): inspection



At the MAC layer, this frame is broadcast. This means it would be protected using the shared GTK and accepted by all clients.

At the IP layer, the packet is targeting a specific victim.

AirSnitch Attack (9): conclusions & countermeasures

- **Improve Network Isolation:** use VLANs to separate untrusted clients (e.g. guest networks). The idea is to isolate at network level, not only at AP logic.
- **Spoofing Prevention:** detect and reject spoofed MAC addresses. This prevent attacker from impersonating victim or gateway.
- **Standardize Client Isolation:** define stronger and consistent isolation guarantees. Ensure APs enforce isolation across all layers.

AirSnitch exploits design assumptions, not cryptographic weaknesses.
Defences must redesign isolation and trust models.

Quantitative Metrics & Real Impact

KRACK	FragAttacks	AirSnitch
• Affects all WPA2 implementations • Impact depends on protocol used. Formally verified protocols can fail in practice.	• 53 out 68 devices vulnerable (78%) • Multiple flaws (fragmentation, aggregation) System design issues across devices.	• 100% success rate in MitM test (5/5) • Packet loss: 1.7% - 7% depending on distance The attack is stable and realistic in practice.

Thank You !

UNIVERSITÀ
DI PISA